

Python best Practice:

→ Python Enhancement Proposal (PEP)

A design document providing information to the python community or describing a new feature for python or its processes or environment

→ on python official website you can open it :

Python.org → documentation → developers guide → quick links → pep

Meta-PEPs (PEPs about PEPs or Processes)

	PEP	Title	Authors
P	1	PEP Purpose and Guidelines	Warsaw, Hylton, Goodger, Coghlan
P	2	Procedure for Adding New Modules	Cannon, Faassen
P	4	Deprecation of Standard Modules	Cannon, von Löwis
P	5	Guidelines for Language Evolution	Prescod
P	6	Bug Fix Releases	Aahz, Baxter
P	7	Style Guide for C Code	GvR, Warsaw
P	8	Style Guide for Python Code	GvR, Warsaw, Coghlan
P	10	Voting Guidelines	Warsaw
P	11	CPython platform support	von Löwis, Cannon
P	12	Sample Structured Text PEP Template	Goodger, Warsaw, Cannon

→ this table on the page shows all the versions along with the content it consist of.

→ Pep8 is the coding style of python code and by styling we mean styling and not program correctness

→ pep 8 allows us to write th puthon code in an efiecient way such that your code will be easier to understand and read and easy maintain

→ PEP8 helps :

1. Formatting : lay-out of code and comments
2. Whitespace and punctuation : indentation , quotation , operators ,etc
3. Naming : Classes , Functions and variables
4. Tools : that helps you out if you get it wrong

→ "A Foolish Consistency is the Hobgoblin of Little Minds" - Ralph Waldo Emerson

→ the creator of python realised very well that code is red much more often than its written and the ultimate goal of pep8 is to improve the readability of code and make it consistent as possible . Sometime you have to be inconsistent

→ Tabs or Spaces?

Spaces are the preferred indentation method.

Tabs should be used solely to remain consistent with code that is already indented with tabs.

Python 3 disallows mixing the use of tabs and spaces for indentation.

Python 2 code indented with a mixture of tabs and spaces should be converted to using spaces exclusively.

When invoking the Python 2 command line interpreter with the -t option, it issues warnings about code that illegally mixes tabs and spaces. When using -tt these warnings become errors. These options are highly recommended!

→ Maximum Line Length

Limit all lines to a maximum of 79 characters.

For flowing long blocks of text with fewer structural restrictions (docstrings or comments), the line length should be limited to 72 characters.

Limiting the required editor window width makes it possible to have several files open side

→ Blank Lines :

Surround top-level function and class definitions with two blank lines.

Method definitions inside a class are surrounded by a single blank line.

Extra blank lines may be used (sparingly) to separate groups of related functions. Blank lines may be omitted between a bunch of related one-liners (eg. a set of dummy implementations).

Use blank lines in functions, sparingly, to indicate logical sections.

Python accepts the control-L (i.e. ^L) form feed character as whitespace; Many tools treat these characters as page separators, so you may use them to separate pages of related sections of your file. Note, some editors and web-based code viewers may not recognize control-L as a form feed and will show another glyph in its place.

→ Imports

Imports should usually be on separate lines, e.g.:

Yes:

```
import os
```

```
import sys
```

No:

```
import os, sys
```

It's okay to say this though:

```
from subprocess import Popen, PIPE
```

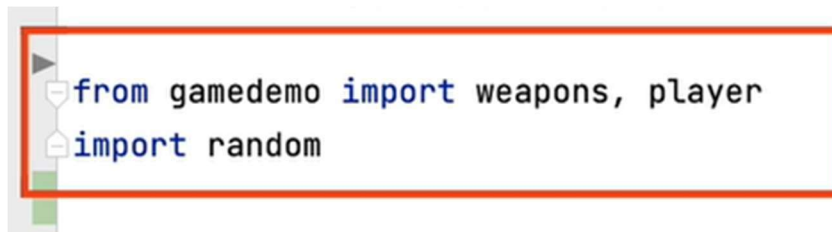
Imports are always put at the top of the file, just after any module comments and docstrings, and before module globals and constants.

These imports are always supposed to be on different lines :

Three groups , separated by blank lines

1. Standard library
2. Third-party library
3. Local application/modules

Sometimes the ide doesn't warn us about the order of import



This is wrong to make this right , we will go to code menu and select optimize imports

→Layout :

1. Indentation : 4 spaces per level
2. Max-lines length 79 characters
3. 2 Lines between top-level functions and classes
4. 1 line between methods inside a class
5. Use of spaces in expression

➔ Ide helps to adjust space

➔ Naming :

1. Modules starts with lower case letters
2. Classes – capitalized naming
3. Function lowercase with underscore for snake casing
4. Constants – all_caps
5. Non public names starts with underscore

➔ Documentation :

1. Docstring for al public modules , functions and classes and methods
2. Guidelines for documentation are in PEP257

➔ Tools used for checking your code :

1. **Pylint** : it gives us control so you can define the code style for your project . is a comprehensive tool that checks for code correctness, style, and complexity.

What is Pylint?

- Pylint is a static code syver for Python 2 or 3. The latest version supports Python 3.7.2 and above.
 - Pylint analyses your code without actually running it. It checks for errors, enforces a coding standard, looks for me, and can make suggestions about how the code could be refactored. Pylint can infer actual values from your code using its internal code representation (astroid). If your code is import logging as argparse, Pyllint will know that argparse.error(...) is in fact a logging call and not an argparse call
 - Pylint is highly configurable and permits to write plugins in order to add your own checks (for example, for internal libraries or an internal rule). Pylint also has an ecosystem of existing plugins for popular frameworks and third party libraries.
2. **Flake8** is a faster tool that focuses specifically on style guide enforcement (PEP8) and common errors.

➔ While both are popular, newer tools like **Ruf** are sometimes used to replace both because they are significantly faster.

➔ Tool that can fix your code automatically :

1. **Black** : it automatically fixes all the styling problems in the code

➔ Standard checkers that are checkedby default :

1. **--analyse-fallback-blocks**: This setting tells Pylint whether to analyze code inside try/except blocks intended to manage compatibility between Python 2 and Python 3.
2. ⓘ **--confidence**: This option filters the reports to only show warnings that meet a specific level of certainty, such as HIGH or INFERENCE.
3. ⓘ **--disable**: This flag is used to turn of specific messages, categories, or entire checkers by their ID or name

➔ Basic checkers in pylint :

1. ⓘ **--argument-naming-style**: Sets the naming convention for function/method arguments.

- a. **Default:** snake_case
- 2. ? **--argument-rgx:** A regular expression for argument names. If provided, it overrides the naming style above.
 - a. **Default:** None
- 3. ? **--attr-naming-style:** Sets the naming convention for class attributes.
 - a. **Default:** snake_case
- 4. ? **--attr-rgx:** A regular expression for attribute names. Overrides the naming style if set.
 - a. **Default:** None
- 5. ? **--bad-names:** A list of specific variable names that are always prohibited.
 - a. **Default:** ('foo', 'bar', 'baz', 'toto', 'tuto', 'tata')
- 6. ? **--bad-names-rgxs:** A list of regular expressions. Any variable name matching these will be flagged as "bad."
 - a. **Default:** (Not fully visible, but usually empty)

➔ **Classes Checker in Pylint**, a static code analysis tool for Python. It lists configuration options used to customize how Pylint evaluates class definitions and member access.

- 1. ? **--check-protected-access-in-special-methods**
 - a. **Purpose:** Toggles warnings when protected attributes (starting with `_`) are accessed inside "dunder" methods like `__str__` or `__repr__`.
 - b. **Default:** False
- 2. ? **--defining-attr-methods**
 - a. **Purpose:** Defines which methods are allowed to create new instance attributes without triggering a "non-defined-at-init" warning.
 - b. **Default:** `__init__`, `__new__`, `setup`, `__post_init__`
- 3. ? **--exclude-protected**
 - a. **Purpose:** A "whitelist" for protected member names that should never trigger access warnings (common in namedtuples or dataclasses).
 - b. **Default:** `_asdict`, `_fields`, `_replace`, `_source`, `_make`
- 4. ? **--valid-classmethod-first-arg**
 - a. **Purpose:** Sets the expected name for the first argument of a `@classmethod`.
 - b. **Default:** `cls`
- 5. ? **--valid-metaclass-classmethod-first-arg**
 - a. **Purpose:** Sets the expected name for the first argument of class methods within a metaclass.
 - b. **Default:** `cls`

➔ Design Checkers in pylint : **Design Checker** section of the **Pylint 2.16.0-dev documentation**. It lists various configuration options used to customize how Pylint evaluates code design.

1. `--exclude-too-few-public-methods`: A list of regular expressions for class names to ignore when counting public methods (related to warning R0903).
2. `--ignored-parents`: Qualified class names to ignore when counting class parents (related to warning R0901).
3. `--max-args`: Maximum number of arguments allowed for a function or method. (Default: 5)
4. `--max-attributes`: Maximum number of attributes allowed for a class. (Default: 7)
5. `--max-bool-expr`: Maximum number of boolean expressions allowed in an if statement. (Default: 5)
6. `--max-branches`: Maximum number of branches allowed for a function or method body. (Default: 12)

➔ To use pylint you have to install pylint

➔ Code to check by using pylint : `pylint <package name>`

➔ This will give us all the problems related to styling

➔ Generating rc file the configuration file that helps us create a configuration file , where you can store your output data



```
[python-code-quality]reindert@CodeSensei-Mac-mini:~/dev/ps/d
- pylint --generate-rcfile > pylintrc
```

➔ **Purpose**: This command creates a new file named `pylintrc` in your current directory, populated with all default Pylint settings and configuration options.

➔ **Customization**: You can edit the resulting `pylintrc` file to enable or disable specific checks, adjust thresholds, and customize the linting rules to fit your project's coding standards

➔ **Usage**: Pylint will automatically look for this file in your project root to apply these settings during future linting sessions.

➔ **This rc command is same everywhere**

- Flake 8 : gives faster and simple checks

Flake8

Flake8 is a wrapper around these tools:

- PyFlakes
- pycodestyle
- Ned Batchelder's McCabe script

Flake8 runs all the tools by launching the single `flake8` command. It displays the warnings in a per-file, merged output.

It also adds a few features:

- files that contain this line are skipped:

```
# flake8: noqa
```

- lines that contain a `# noqa` comment at the end will not issue warnings.
- you can ignore specific errors on a line with `# noqa: <error>`, e.g., `# noqa: E234`. Multiple codes can be given, separated by comma. The `noqa` token is case insensitive, the colon before the list of codes is required otherwise the part after `noqa` is ignored

- Flakes8 checks for general codes s potential problems but not pep8 style problems and McCabe scripts check for unnecessary complexity .
- This overlaps with pylint . the main point here is flakes8 does a smaller set of things than pylint
- Install flakes8
- Then run flakes8 wiioth python file, here we mostly get the error pylint doesn't show

```
gamedemo/game.py:15:4: E111 indentation is not a multiple of 4
gamedemo/game.py:16:9: E117 over-indented
gamedemo/game.py:25:4: E111 indentation is not a multiple of 4
gamedemo/game.py:26:7: E111 indentation is not a multiple of 4
gamedemo/game.py:27:7: E111 indentation is not a multiple of 4
gamedemo/game.py:28:7: E111 indentation is not a multiple of 4
gamedemo/game.py:29:10: E111 indentation is not a multiple of 4
gamedemo/game.py:32:10: E111 indentation is not a multiple of 4
gamedemo/game.py:35:10: E111 indentation is not a multiple of 4
gamedemo/game.py:36:10: E111 indentation is not a multiple of 4
gamedemo/game.py:37:10: E111 indentation is not a multiple of 4
gamedemo/game.py:38:10: E111 indentation is not a multiple of 4
gamedemo/game.py:39:7: E111 indentation is not a multiple of 4
gamedemo/game.py:40:18: E127 continuation line over-indented for visual indent
gamedemo/game.py:42:1: E305 expected 2 blank lines after class or function definition, found 1
[python-code-quality]reindert@CodeSensei-Mac-mini:~/dev/ps/demos/python-code-quality|tools
```

- If you prefer more complete checks for pep8 error , it is perffered to use flake8 , but if we want more complex flexible tool,that can run checks for more different types of code smells , we would preffers using pylint

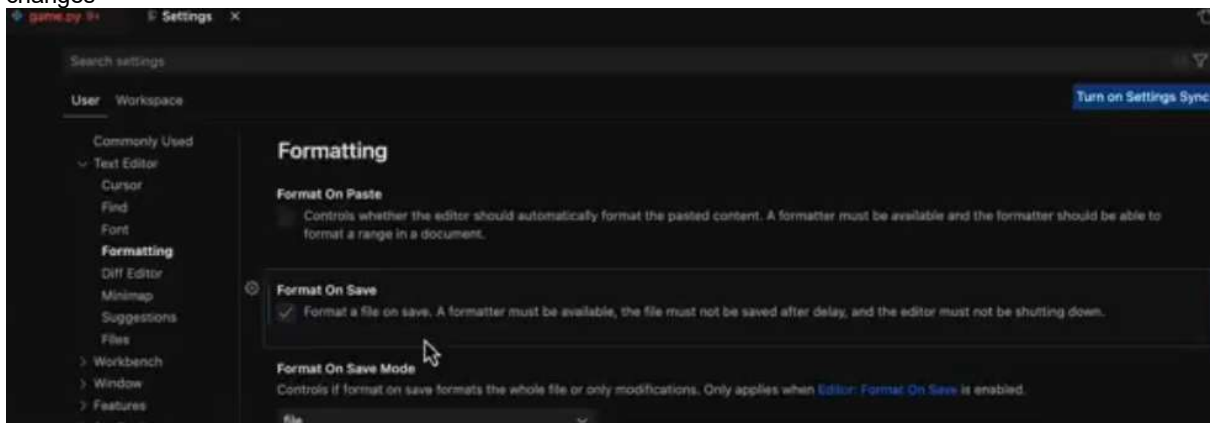
```
[flake8]
extend-ignore = E203
exclude = .git,__pycache__,docs/source/conf.py,old,build,dist
max-complexity = 10
```

- Black :
- It is a tool can has taken over the world of python from the past few years
- This is differnta as it checnge the file diorectly so that it follows pep8
- Start by installing black : pip install black

```
black gamedemo
reformatted gamedemo/__init__.py
reformatted gamedemo/player.py
reformatted gamedemo/weapons.py
reformatted gamedemo/game.py

All done! 🎉 🍌 🎉
```

- ➔
- ➔ Black call itself uncompromising code formatter
 - **Uncompromising Style:** Black enforces a strict, consistent style, freeing developers from manual formatting decisions.
 - **Speed and Determinism:** It offers fast formatting and ensures code looks identical across different projects.
 - **Small Diffs:** The tool helps make code reviews more efficient by producing the smallest possible changes when formatting code
- ➔ Black fixes all to problems except line lengths :
- ➔ On vscode the option for automatic checking is not enabled , if you would like to use it , you have to got to setting and enable linting under python tab
- ➔ After enabling , vscode will automatically install pylint
- ➔ Under vscode we can enable format after save so that vscode automatically saves the code after every changes



- ➔ Tools: Checkers
 - pycodestyle:
 - -check PEP8 rules
 - flake8
 - -PEP8
 - -code smells
 - -complexity
 - pylint
 - -very flexible, can check many things
- ➔ ☐ **pycodestyle:** Specifically designed to check Python code against the PEP 8 style guide, which dictates how code should be formatted for readability. [\[1\]](#)
- ➔ ☐ **flake8:** A wrapper tool that combines multiple checks into one, including checking PEP 8 compliance, looking for "code smells" (potential design problems), and analyzing code complexity. [\[1, 2\]](#)
- ➔ ☐ **pylint:** A very flexible tool that performs a wide range of checks, including enforcing coding standards, detecting errors, and suggesting refactoring opportunities

Documenting Your Project

Documenting project by using as tool called sphinx

Sphinx is used to generate the documentation for the python language and the standard library and it also transforms the docstring in your code into beautiful html

Sphinx also uses a markup format called restructured text for laying out text

🔗 **PEP 257** defines official standards for writing docstrings in Python to improve code clarity and consistency.

🔗 A docstring must be the **first statement** within a module, function, class, or method definition.

🔗 These strings are stored automatically in the object's `__doc__` attribute.

🔗 The guidelines cover best practices for both **one-line** and **multi-line** docstrings. [[1](#), [2](#), [3](#)]

standard Python docstring conventions. Here is a brief summary of the rules presented:

- **Syntax:** Use triple double quotes ("""") to enclose the docstring.
- **Structure:** Start with a summary phrase that ends with a period.
- **Methods:** Clearly define what the function or method returns.

Running sphinx :

- ➔ Install sphinx
 - ➔ Mkdir docs for creating a directory named as docs , Run command from in there
 - ➔ The sphinx-quickstart command initializes the project structure and creates necessary configuration files, including conf.py.
 - ➔ Running make html uses the generated files to build the final documentation website in HTML format
-
- ➔ **reStructuredText (reST)**, a plain-text markup language commonly used in the Python community for documentation (e.g., Sphinx).

Key Features Mentioned:

- **Plain-text based markup:** It is designed to be highly readable in its raw form.
- **Paragraph separation:** Paragraphs are defined by a single blank line between blocks of text.
- **Indentation matters:** Like Python, indentation is used to indicate structure, such as block quotes or nested lists.

<pre>===== A Chapter ----- A Section A Subsection</pre>	<pre>* Bulleted list item one * Bulleted list item two + Milk + Butter 'Link text <http://example.com/>'_ http://www.example.com</pre>
---	--

Type hints :

Applying type hints in a project

Type hints are used for define what type of data can a variable store .

```
def __init__(self, name: str, weapon: Weapon, health: int) -> None:
```

It initializes a new object instance with specific attributes: a name (string), a weapon (Weapon type), and health (integer).

This helps in static type checking when I am define it warns you regarding the type of data it stores :

```
l = game(
    Player("The Knight", 100, Sword()),
    Player("The Dragon", 500, FireBreath())
```

Static vs. dynamic typing :

The function of type hints is to give early warning before excecuting the program and it doesn't mean that you can just give int value but it represents the for this function we excepts int values .

```
def average(a: int, b: int, c: int) -> float:
    return (a + b + c) / 3
```

```
print(average(1, 5, 7))
print(average("this", "still", "runs"))
```

Classes as Types :

We can add classes as types , all the classes are type

1. Built-in Classes : These are core types provided by Python itself.
 - a. **Data Types:** int (integers), float (decimals), str (text), bool (True/False).
 - b. **Collections:** list (ordered items), dict (key-value pairs), tuple (fixed sequences).
2. Classes you wrote : These are **User-Defined Classes**. You create them using the class keyword to group data and functions that belong together (e.g., a Car class or a User class).
3. Classes you import : These are classes defined in **external modules or libraries**. For example, if you import datetime, you gain access to the datetime.date class.

Use: <http://mypy.readthedocs.io>

It provides resources for adding type hints to Python programs to ensure correct variable and function usage.

what is type inference in python?

- ➔ Type inference in Python refers to the process by which programming tools, IDEs, and static type checkers automatically determine the data type of a variable, expression, or function return value based on its context and value, without requiring explicit type annotations

Notes : all the classes in python are types but there are certain types that are not classes

Example :

1. Union : int | str or float | None
2. Any

```
def max_avg(d: dict[Any, list[float]]) -> float:
    return max(sum(lst)/len(lst) for lst in d.values())
```

- Finally : Mypy a type checker four command line :

To use mypy we need install mypy and just like other chckers we need to run mypy with the file we would like to check

```
def __init__(self):
    # The number of attacks we will have to wait until
    self._cooldown = 0
```

```
def attack(self) -> tuple[int, str] | None:
    if self._cooldown:
        self._cooldown -= 1
        return None
```

```
        dice_roll = random.randrange(0, 20)
        if dice_roll > 16:
            dmg = 4 * dice_roll
            self._cooldown = 2
            sound = "Boom! Dragon Fire!"
        else:
            dmg = 0
            sound = "There's lots of smoke but no fire."
        return dmg, sound
```

This return satatements in attack at different is considered correct by pycharm as the return type was set to be tuple[int , str] | None ,

But when we are checking the same by using mypy it return error saying that this is incompatible

```
python-code-quality@reindert@CodeSensei-Mac-mini:~/dev/ps/demos/python-code-quality(master)
$ mypy gamedemo
gamedemo/weapons.py:48: error: Return type "Optional[Tuple[int, str]]" of "attack" incompatible with return type "Tuple[int, str]" in supertype "Weapon"  [override]
Found 1 error in 1 file (checked 4 source files)
```

By enabling mypy in setting of vscode it will work similarly as that of pylint and black